Graph Traversal (DFS and BFS) for Game Map Exploration

Lab Report: Data Structures and Algorithms

1. Aim

To represent game locations as a graph using an adjacency list, and to perform Depth First Search (DFS) and Breadth First Search (BFS) from a user-selected starting vertex. The program displays traversal orders, BFS levels, and the reachable and unreachable vertices.

2. Introduction

A game map can be modelled as a graph. Each location (room, town, checkpoint) is a **vertex**, and each path between two locations is an **edge**. Graph traversal answers practical questions: which locations can the player reach from the start, how many moves away is each one, and which areas are cut off from the rest of the map.

This program stores the map as an **adjacency list** (an array of vectors), which uses memory proportional to the number of vertices plus edges and is well suited to sparse maps. Paths are undirected, so every edge is added in both directions.

3. Theory

Depth First Search (DFS) explores as far as possible along one path before backtracking. It is implemented recursively and marks each vertex as visited. It is useful for checking connectivity and exploring a map branch by branch.

Breadth First Search (BFS) explores all neighbours of a vertex before moving to the next level, using a queue. Because it expands outward in layers, the level of each vertex equals its shortest distance (in number of edges) from the start. This makes BFS ideal for finding the fewest moves between locations.

Complexity. With V vertices and E edges, both DFS and BFS run in O(V + E) time using an adjacency list. Space is O(V) for the visited and level arrays, plus O(V + E) for the graph itself.

4. Algorithm

1. Create the graph using an adjacency list.
2. Read the number of vertices and edges, then add each edge in both directions.
3. Select the starting vertex.
4. Perform DFS from the start and display the visiting order.
5. Perform BFS from the start. Set the start level to 0 and each newly discovered vertex's level to its parent's level + 1.
6. Display all vertices reached by BFS together with their levels, and the reachable count.
7. Display the vertices that were never reached (unreachable).
8. Stop.

5. C++ Program

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

vector<int> graph[10];
bool visited[10];
int countVisited = 0;

void DFS(int start) {
    visited[start] = true;
    cout << start << " ";
    countVisited++;

    for (int next : graph[start]) {
        if (!visited[next])
            DFS(next);
    }
}

void BFS(int start, int n) {
    bool seen[10] = {false};
    int level[10];

    for (int i = 0; i < n; i++)
```

```
        level[i] = -1;

    queue<int> q;
    q.push(start);
    seen[start] = true;
    level[start] = 0;

    cout << "\nBFS Traversal: ";

    while (!q.empty()) {
        int current = q.front();
        q.pop();
        cout << current << " ";

        for (int next : graph[current]) {
            if (!seen[next]) {
                seen[next] = true;
                level[next] = level[current] + 1;
                q.push(next);
            }
        }
    }

    int reachable = 0;
    cout << "\nReachable vertices and BFS levels:\n";

    for (int i = 0; i < n; i++) {
        if (seen[i]) {
            cout << i << " : Level " << level[i] << "\n";
            reachable++;
        }
    }

    cout << "Reachable vertex count: " << reachable;

    cout << "\nUnreachable vertices: ";
    bool found = false;

    for (int i = 0; i < n; i++) {
        if (!seen[i]) {
            cout << i << " ";
            found = true;
        }
```

```
    }

    if (!found)
        cout << "None";
}

int main() {
    int n, e, u, v, start;

    cout << "Enter number of vertices: ";
    cin >> n;

    cout << "Enter number of edges: ";
    cin >> e;

    cout << "Enter edges (u v):\n";

    for (int i = 0; i < e; i++) {
        cin >> u >> v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    cout << "Enter starting vertex: ";
    cin >> start;

    cout << "DFS Traversal: ";
    DFS(start);

    cout << "\nDFS visited count: " << countVisited;

    BFS(start, n);

    cout << "\n";
    return 0;
}
```

6. Sample Input and Output

The sample map has 6 locations (0 to 5) and 5 paths: 0-1, 0-2, 1-3, 2-3 and 3-4. Location 5 has no paths, so it is cut off from the rest of the map.

Input

```
Enter number of vertices: 6
Enter number of edges: 5
Enter edges (u v):
0 1
0 2
1 3
2 3
3 4
Enter starting vertex: 0
```

Output

```
DFS Traversal: 0 1 3 2 4
DFS visited count: 5
BFS Traversal: 0 1 2 3 4
Reachable vertices and BFS levels:
0 : Level 0
1 : Level 1
2 : Level 1
3 : Level 2
4 : Level 3
Reachable vertex count: 5
Unreachable vertices: 5
```

Explanation. DFS from 0 goes to 1, then 3, then back through 3′s neighbour 2, and finally to 4, giving 0 1 3 2 4. BFS visits 0, then its neighbours 1 and 2 (level 1), then 3 (level 2), then 4 (level 3). Vertex 5 has no connection to the start, so it is reported as unreachable.

7. Observations

- DFS and BFS visit the same set of vertices (everything in the start vertex′s connected component) but in different orders.
- BFS levels give the minimum number of moves from the start to each location.
- Vertices that never get a level (-1) are exactly the unreachable ones.
- The program uses fixed arrays of size 10, so it supports at most 10 vertices (numbered 0 to 9). Increase the array sizes for larger maps.
- Vertex numbers are assumed to be valid (0 to n-1); the program does not validate input.

8. Conclusion

The program successfully represents game locations using a graph and performs DFS and BFS from a user-selected starting vertex. It displays traversal orders, identifies reachable and unreachable vertices, counts reachable vertices, and calculates BFS levels. Thus, it demonstrates how graph traversal algorithms can be used to explore game maps and determine connectivity between locations.